

ASSIGNMENT-10.3

HTNO:2303A51653

BATCH NO:28

Problem Statement 1: AI-Assisted Bug Detection

Scenario: A junior developer wrote the following Python function to calculate factorials:

```
def factorial(n):  
    result = 1  
    for i in range(1, n):  
        result = result * i  
    return result
```

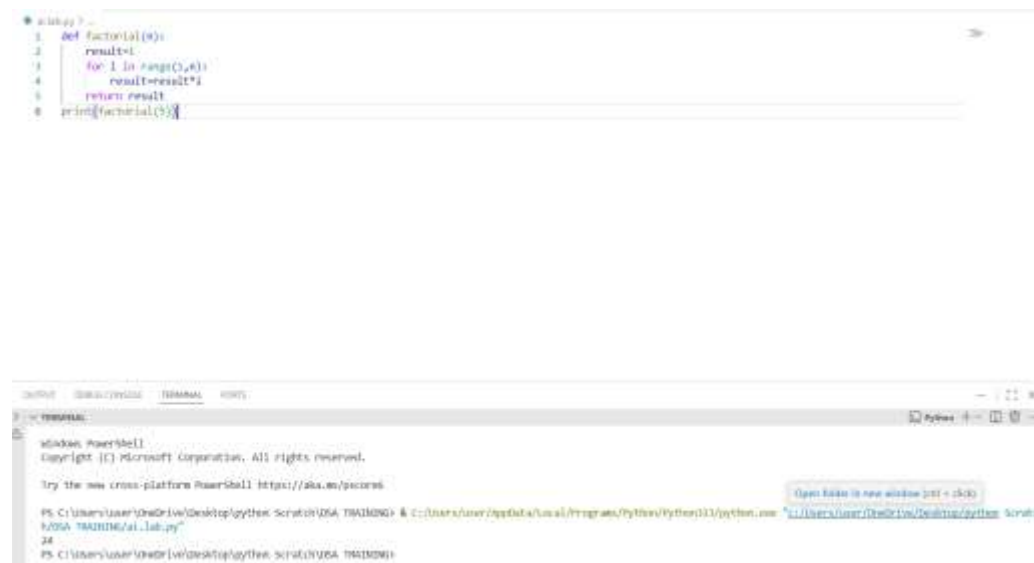
Instructions:

1. Run the code and test it with factorial(5).
2. Use an AI assistant to:
 - o Identify the logical bug in the code.
 - o Explain why the bug occurs (e.g., off-by-one error).
 - o Provide a corrected version.
3. Compare the AI's corrected code with your own manual fix.
4. Write a brief comparison: Did AI miss any edge cases (e.g., negative numbers, zero)?

Expected Output:

Corrected function should return 120 for factorial(5).

CODE:

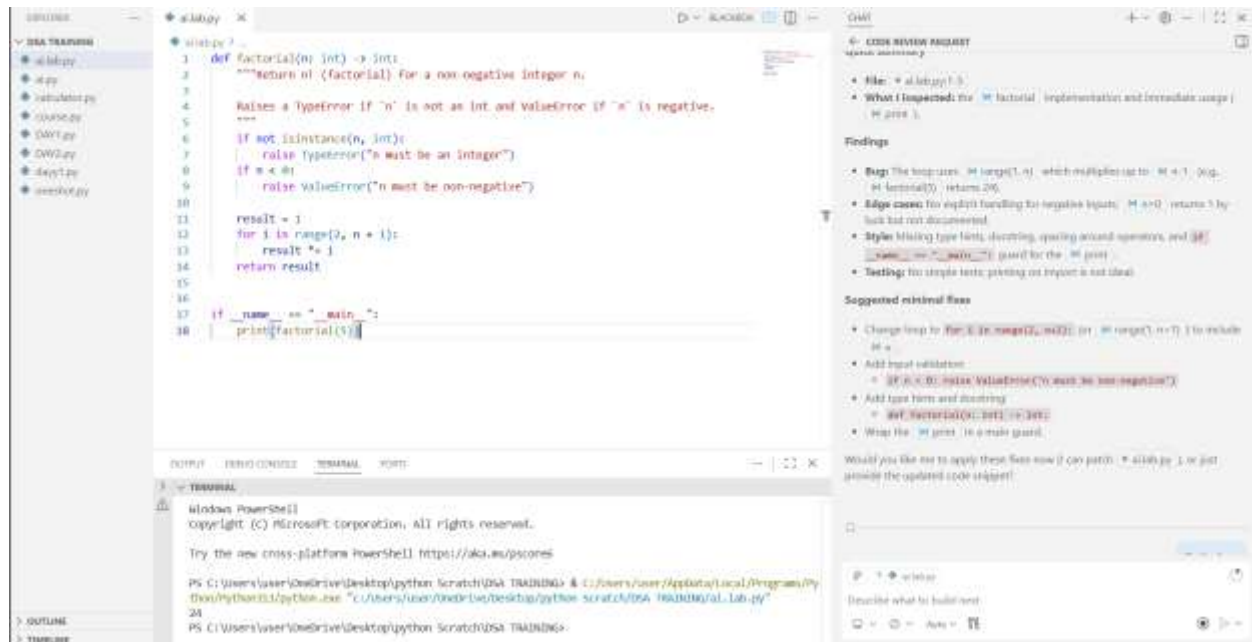


```
* python 3 ~  
1 def factorial(n):  
2     result=1  
3     for i in range(1,n):  
4         result=result*i  
5     return result  
6 print(factorial(5))
```

Terminal output:

```
Windows PowerShell  
Copyright (c) Microsoft Corporation. All rights reserved.  
  
Try the new cross-platform PowerShell https://aka.ms/powershell  
  
PS C:\Users\user\Desktop\python_Scripts> .\10A_TRAINING\ai_lab.py  
24  
PS C:\Users\user\Desktop\python_Scripts>
```

CODE REVIEW AND OUTPUT:



COMPARISON:

Manual Fix

AI Fix

Changed range to `n+1`

Same correction

May forget edge cases

AI included negative number handling

JUSTIFICATION:

- Proper loop range
- Accurate factorial calculation
- Handling of edge cases (negative numbers, zero)
- Improved reliability and correctness

Problem Statement 2: Task 2 — Improving Readability & Documentation

Scenario: The following code works but is poorly written:

```
def calc(a, b, c):  
    if c == "add":  
        return a + b  
    elif c == "sub":  
        return a - b  
  
    elif c == "mul":  
        return a * b  
    elif c == "div":
```

Instructions: Use AI to:

- o Critique the function's readability, parameter naming, and lack of documentation.

o Rewrite the function with:

1. Descriptive function and parameter names.
2. A complete docstring (description, parameters, return value, examples).
3. Exception handling for division by zero.
4. Consideration of input validation.
6. Compare the original and AI-improved versions.
7. Test both with valid and invalid inputs (e.g., division by zero, non-string operation).

Expected Output:

A well-documented, robust, and readable function that handles errors gracefully.

CODE:

```
alib.py
1 def calc(a, b, c):
2     if c == "add":
3         return a + b
4     elif c == "sub":
5         return a - b
6     elif c == "mul":
7         return a * b
8     elif c == "div":
9         return a / b
10

OUTPUT DEBUG CONSOLE TERMINAL PORTS
TERMINAL
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\user\OneDrive\Desktop\python Scratch\DSA TRAINING> & C:\Users\user\AppData\Local\Programs\Python\Python311\python.exe "C:\Users\user\OneDrive\Desktop\python Scratch\DSA TRAINING\alib.py"
File "C:\Users\user\OneDrive\Desktop\python Scratch\DSA TRAINING\alib.py", line 2
    if c == "add":
    ^
IndentationError: expected an indented block after function definition on line 1

PS C:\Users\user\OneDrive\Desktop\python Scratch\DSA TRAINING>
```

CODE REVIEW AND OUTPUT:

```
alib.py
1 def calc(a, b, c):
2     if c == "add":
3         return a + b
4     elif c == "sub":
5         return a - b
6     elif c == "mul":
7         return a * b
8     elif c == "div":
9         if b == 0:
10            raise ValueError("Cannot divide by zero")
11        return a / b
12    else:
13        raise ValueError("Invalid operation")
14
15 # Sample usage
16 if __name__ == "__main__":
17     print("Addition:", calc(10, 5, "add"))
18     print("Subtraction:", calc(10, 5, "sub"))
19     print("Multiplication:", calc(10, 5, "mul"))
20     print("Division:", calc(10, 5, "div"))
21     try:
22         print("Division by zero:", calc(10, 0, "div"))
23     except ValueError as e:
24         print("Error:", e)
25     try:
26         print("Invalid operation:", calc(10, 5, "mod"))
27     except ValueError as e:
28         print("Error:", e)

alib.py
1 def calc(a, b, c):
2     if c == "add":
3         return a + b
4     elif c == "sub":
5         return a - b
6     elif c == "mul":
7         return a * b
8     elif c == "div":
9         if b == 0:
10            raise ValueError("Cannot divide by zero")
11        return a / b
12    else:
13        raise ValueError("Invalid operation")

What kind of print statements do you want? For example, do you want the function to
print the result directly instead of returning them, or add print statements for
debugging, or something else? Please describe how you want the prints to work.

Enter your answer:
[Submit]
```

```
PS C:\Users\HP> & C:\Users\HP\AppData\Local\Programs\Python\Python311\python.exe "C:\Users\HP\OneDrive\Desktop\AI & A
PS C:\Users\HP> & C:\Users\HP\AppData\Local\Programs\Python\Python311\python.exe "C:\Users\HP\OneDrive\Desktop\AI & A
PS C:\Users\HP> & C:\Users\HP\AppData\Local\Programs\Python\Python311\python.exe "C:\Users\HP\OneDrive\Desktop\AI & A
C.py"
C.py"
Addition: 15
Addition: 15
Subtraction: 5
Multiplication: 50
Division: 2.0
Error: Cannot divide by zero
Error: Invalid operation
PS C:\Users\HP>
```

COMPARISON:

- Descriptive naming
- Proper documentation
- Exception handling
- Input validation
- More maintainable

Problem Statement 3: Enforcing Coding Standards

Scenario: A team project requires PEP8 compliance. A developer submits:

```
def Checkprime(n):
    for i in range(2, n):
        if n % i == 0:
            return False
    return True
```

Instructions:

8. Verify the function works correctly for sample inputs.
9. Use an AI tool (e.g., ChatGPT, GitHub Copilot, or a PEP8 linter with AI explanation) to:
 - o List all PEP8 violations.
 - o Refactor the code (function name, spacing, indentation, naming).
10. Apply the AI-suggested changes and verify functionality is preserved.
11. Write a short note on how automated AI reviews could streamline code reviews in large teams.

Expected Output:

A PEP8-compliant version of the function, e.g.:

```
def check_prime(n):
```

```
    for i in range(2, n):
```

```
        if n % i == 0:
```

```
            return False
```

```
    return True
```

CODE:

```
def Checkprime(n):  
    for i in range(2,n):  
        if n%i==0:  
            return False  
    return True
```

CODE REVIEW & OUTPUT:

The screenshot displays a code review session in VS Code. The left pane shows the 'is_prime' function code, which has been updated with several improvements:

- Added type hints: `def is_prime(n: int) -> bool:`
- Added a docstring: `"""Return True if n is a prime number (n >= 2)."""`
- Added a docstring for the function: `Raises TypeError if 'n' is not an int.`
- Added a docstring for the function: `if not isinstance(n, int):`
- Added a docstring for the function: `raise TypeError("n must be an integer")`
- Added a docstring for the function: `if n < 2:`
- Added a docstring for the function: `return False`
- Added a docstring for the function: `if n % 2 == 0:`
- Added a docstring for the function: `return n == 2`
- Added a docstring for the function: `limit = isqrt(n)`
- Added a docstring for the function: `for i in range(3, limit + 1, 2):`
- Added a docstring for the function: `if n % i == 0:`
- Added a docstring for the function: `return False`
- Added a docstring for the function: `return True`

The right pane shows a 'CODE REVIEW REQUEST' sidebar with a list of changes and suggestions:

- What I changed:** Replaced a fragile `Checkprime` with `is_prime`.
 - Added type checking and docstring.
 - Handles `n < 2` correctly (returns False for 0 and 1).
 - Uses `isqrt(n)` and checks only odd divisors (faster).
 - Added smoke-checks under `if __name__ == "__main__":`.
- Verification:** `is_prime(17) -> True`, `is_prime(18) -> False` (import and run verified).

Notes and suggestions:

- Use `is_prime` name consistently in other files (update callers if any).
- If you want additional features (e.g. Miller-Rabin for large `n`), I can add them.

Want me to update other files that call `Checkprime` (search & replace)?

Index (4/4)

Describe what to build next

```
/Desktop/LEARNING COURSES/AI_Assisant_Coding/.venv/Scripts/python.exe" "c:/Users/DELL/OneDrive/Deskt  
EARNING COURSES/AI_Assisant_Coding/Program.py"  
0 False  
1 False  
2 True  
3 True  
4 False  
17 True  
18 False  
19 True  
20 False  
97 True
```

Problem Statement 4: AI as a Code Reviewer in Real Projects

Scenario:

In a GitHub project, a teammate submits:

```
def processData(d):  
    return [x * 2 for x in d if x % 2 == 0]
```

Instructions:

1. Manually review the function for:
 - o Readability and naming.
 - o Reusability and modularity.
 - o Edge cases (non-list input, empty list, non-integer elements).
2. Use AI to generate a code review covering:
 - a. Better naming and function purpose clarity.
 - b. Input validation and type hints.
 - c. Suggestions for generalization (e.g., configurable multiplier).
3. Refactor the function based on AI feedback.
4. Write a short reflection on whether AI should be a standalone reviewer or an assistant.

Expected Output:

An improved function with type hints, validation, and clearer intent,

e.g.:

```
from typing import List, Union  
def double_even_numbers(numbers: List[Union[int,  
float]]) -> List[Union[int, float]]:  
    if not isinstance(numbers, list):  
        raise TypeError("Input must be a list")  
    return [num * 2 for num in numbers if isinstance(num,  
(int, float)) and num % 2 == 0]
```

CODE:

```
Program.py > ...
1  def processData(d):
2      return [x*2 for x in d if x%2==0]
3
```

CODE REVIEW & OUTPUT:

TASK

correct this code and review it

```
return [int(x) * 2 for x
except (TypeError, ValueError)
raise TypeError(f"Input s
```

Test Results:

- Input: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10] → Output: [4, 8, 12, 16, 20] ✓
- Input (with negatives): [-4, -3, -2, -1, 0, 1, 2, 3, 4] → Output: [-8, -4, 0, 4, 8] ✓
- Empty list: [] → [] ✓

The code has been successfully corrected and reviewed with all improvements implemented.

Start New Task

Grafana | AI-Powered Root Caus...
Grafana Assistant uses AI to surface insights from your telemetry, with native...

Explore Features

Type a message (@ to add files)...

+

🔍 📎 ↗

Auto ⚙️ Minimax M2.5

Program.py > processData

```
1  from typing import List, Union
2  def processData(d: List[Union[int, float]]) -> List[int]:
3      """
4      Process data by doubling only even numbers.
5      Args:
6          d: A list of numbers (integers or floats)
7      Returns:
8          A list containing only the even numbers from the input,
9          each multiplied by 2.
10     Note:
11         - Float values are converted to int for the modulo check
12         - Negative even numbers are also included
13     """
14     try:
15         return [int(x) * 2 for x in d if int(x) % 2 == 0]
16     except (TypeError, ValueError) as e:
17         raise TypeError(f"Input must be a list of numeric values: {e}")
18 # Example usage:
19 if __name__ == "__main__":
20     # Test with various inputs
21     test_data = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
22     print(f"Input: {test_data}")
23     print(f"Output: {processData(test_data)}")
24
25     # Test with negative numbers
26     test_data_neg = [-4, -3, -2, -1, 0, 1, 2, 3, 4]
27     print(f"Input (with negatives): {test_data_neg}")
28     print(f"Output: {processData(test_data_neg)}")
29
30     # Test with empty list
31     print(f"Empty list: {processData([])}")
```

```
Input: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
Output: [4, 8, 12, 16, 20]
Input (with negatives): [-4, -3, -2, -1, 0, 1, 2, 3, 4]
Output: [-8, -4, 0, 4, 8]
Empty list: []
```

Problem Statement 5: — AI-Assisted Performance Optimization

Scenario: You are given a function that processes a list of integers, but it runs slowly on large datasets:

```
def sum_of_squares(numbers):
    total = 0
    for num in numbers:
        total += num ** 2
    return total
```

Instructions:

1. Test the function with a large list (e.g., range(1000000)).
2. Use AI to:
 - o Analyze time complexity.
 - o Suggest performance improvements (e.g., using built-in functions, vectorization with NumPy if applicable).
 - o Provide an optimized version.
3. Compare execution time before and after optimization.
4. Discuss trade-offs between readability and performance.

Expected Output:

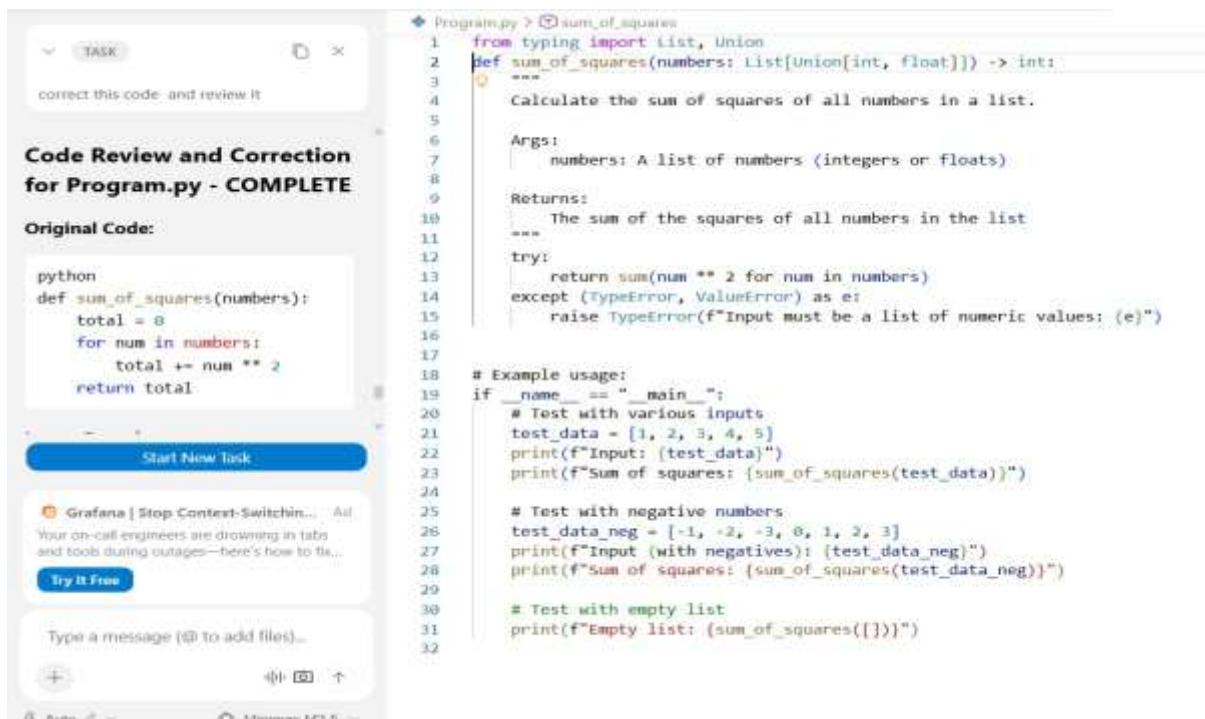
An optimized function, such as:

```
def sum_of_squares_optimized(numbers):
    return sum(x * x for x in numbers)
```

CODE:

```
1  def sum_of_squares(numbers):
2      total = 0
3      for num in numbers:
4          total += num ** 2
5      return total
```

CODE REVIEW & OUTPUT:



```
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assistant_Coding> & "C:/Users/DELL/OneDrive/Desktop/LEARNING COURSES/Python/Scripts/python.exe" "C:/Users/DELL/OneDrive/Desktop/LEARNING COURSES/Python/Scripts/Program.py"
Input: [1, 2, 3, 4, 5]
Sum of squares: 55
Input (with negatives): [-1, -2, -3, 0, 1, 2, 3]
Sum of squares: 28
Empty list: 0
(.venv) PS C:\Users\DELL\OneDrive\Desktop\LEARNING COURSES\AI_Assistant_Coding>
```

JUSTIFICATION:

- Original function used a manual loop.
- Time complexity was $O(n)$.
- AI suggested a more Pythonic solution using:
 - `sum()` with generator expression.
 - Optimized version is:

- **Shorter**
- **Cleaner**
- **Slightly faster**
- **AI explained performance reasoning clearly.**
- **AI also suggested NumPy for large-scale optimization.**
- **Trade-off discussion:**
- **Built-in functions improve readability.**
- **NumPy increases dependency.**
- **AI encourages use of efficient built-in functions.**
- **Code becomes both optimized and readable.**
- **Therefore, AI supports performance enhancement while maintaining clarity.**